



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO

PARADIGMAS DE PROGRAMACIÓN

UNIDAD 2

Facultad de Ingeniería
Universidad Nacional de Cuyo

2026

- 1 Introducción
- 2 Formalidad
- 3 Reglas y Resolución
- 4 Uso

Introducción

Lambda

- Inventado por Alonzo Church como un formalismo matemático para expresar el cómputo por medio de funciones.
- Un cálculo que captura el aspecto computacional de la noción de función (Church [1932-1933, 1940])
- Propiedades generales de las funciones independiente del área particular de problemas.
- Los lenguajes funcionales modernos pueden ser entendidos como un embellecimiento sintáctico del calculo lambda (azucarado sintatico).

Objetivos

- El objetivo es dar una teoría general de las funciones
- Es un sistema formal diseñado para definir funciones, la forma de utilizarlas y la recursión.
- Es utilizado como un fundamento de lenguajes de programación porque aporta una sintaxis básica de programación, la semántica para el concepto e función en la transformación de argumentos en resultados y una forma de definir primitivas de programación.

Calculo Lambda

Lambda es:

- Un lenguaje de programación pequeño que consiste en transformaciones simples.
 - Definir funciones
 - Sustituir entidades

Formalidad

¿Para qué sirve el Cálculo Lambda?

- Marco formal para estudiar propiedades de procesos de cómputo, programas, lenguajes, entre otros
- El cálculo lambda sirve para entender y formalizar la computación y las funciones, y es la “gramática” teórica detrás de muchas ideas que después aparecen en Haskell y en lenguajes modernos.

Calculo Lambda

- En el cálculo lambda, una expresión o término se define recursivamente a través de las siguientes reglas de formación:
- Toda entidad que podemos llamar "variable" es un término:
 $x, y, z, u, v, w, x_1, x_2, y_9, \dots$
- Si $E1$ es un término y x es una variable, entonces $(\lambda x.E1)$ es un término (llamado una abstracción lambda).
- Si $E1$ y $E2$ son términos, entonces $(E1E2)$ es un término (llamado una aplicación lambda).

NOTA

NADA MAS ES UN TERMINO

Lambda

El concepto central en calculo lambda es la definición de su “expresión”.

$$\langle \text{expresión} \rangle := \langle \text{variable} \rangle \mid \langle \text{abstracción} \rangle \mid \langle \text{aplicación} \rangle$$

$$\langle \text{abstracción} \rangle := \lambda \langle \text{variable} \rangle . \langle \text{expresión} \rangle$$

$$\langle \text{aplicación} \rangle := \langle \text{expresión} \rangle \langle \text{expresión} \rangle$$

Ejemplo

abstracción := $\lambda x.x$

aplicación := $(\lambda x.x)y$

expresión := $(\lambda x.x)((\lambda x.x)y)$

NOTA: En esta sintaxis no existe el concepto de $\langle \text{nombre} \rangle$ o $\langle \text{constante} \rangle$

Términos lambda

- v es una entidad o también llamado variable
- **Abstracción (función):** $\lambda v. E1$ representa una función anónima que toma v como parámetro y cuyo cuerpo es $E1$. Cuando la función se aplica a un argumento, las ocurrencias libres de v en $E1$ se sustituyen por dicho argumento.
- **Aplicación (llamada a función):** $(E1 E2)$ se denomina *aplicación* y representa la llamada a la expresión $E1$ (interpretada como función) pasando $E2$ como argumento. En particular, si $E1$ es una abstracción, entonces:

$$((\lambda v. E) E2) \rightarrow E[v := E2],$$

es decir, el resultado es el cuerpo E donde cada ocurrencia libre de v se reemplaza por $E2$.

Cálculo Lambda

- El cálculo lambda se puede ver como un lenguaje de definición de funciones cuyos únicos tipos de datos primitivos son las funciones
- no hay números ni cadenas de caracteres
- incluye un mecanismo de evaluación de las funciones basado en sustitución de variables por expresiones. En particular, todas las funciones tienen un argumento que representa a su vez otra función y sus imágenes son a su vez funciones.

Cadenas de ejemplos

x

(xy)

$((xzy)x)$

$(\lambda x.x)$

$((\lambda x.x)y)$

$(\lambda z.(\lambda x.y))$

$((x(\lambda z.z))z)$

Convenciones

- Por convención se suelen omitir los paréntesis externos, ya que no cumplen ninguna función de desambiguación.
- Por ejemplo se escribe $(\lambda z.z)z$ en vez de $((\lambda z.z)z)$, y se escribe $x(y(zx))$ en vez de $(x(y(zx)))$.
- Además se suele adoptar la convención de que la aplicación de funciones es asociativa hacia la izquierda.
- Esto quiere decir, por ejemplo, que:
 - $xyzz$ debe entenderse como $((xy)z)z$
 - $(\lambda z.z)yzx$ debe entenderse como $((((\lambda z.z)y)z)x)$.

,

Convenciones Sintácticas

Convenciones sintácticas para hacer más sencillas las λ - expresiones

- 1 La aplicación va a ser asociativa por la izquierda:

$$(((MN)P)Q) \rightarrow MNPQ$$

- 2 La abstracción es asociativa por la derecha:

$$(\lambda x. (\lambda y. M)) \rightarrow \lambda x. \lambda y. M$$

- 3 La aplicación es prioritaria sobre la abstracción:

$$(\lambda x. (MN)) \rightarrow \lambda x. MN$$

- 4 Se puede suprimir símbolos λ en abstracciones consecutivas:

$$\lambda x. \lambda y. \dots \lambda z. M \rightarrow \lambda xy \dots z. M$$

Por qué usamos convenciones

- Las expresiones con paréntesis completos se vuelven largas.
- Las **convenciones sintácticas** permiten escribir lo mismo de forma más corta, pero con una interpretación fija (sin ambigüedad).
- Objetivo: **ahorrar paréntesis** sin perder precisión.

Convención 1: aplicación asociativa por la izquierda

$$(((MN)P)Q) \rightarrow MNPQ$$

Ejemplos:

$$(((f\ x)\ y)\ z) \rightarrow f\ x\ y\ z$$

$$((g\ a)\ b) \rightarrow g\ a\ b$$

- Lectura: $MNPQ$ siempre significa $(((MN)P)Q)$.

Convención 2: abstracción asociativa por la derecha

$$(\lambda x.(\lambda y.M)) \rightarrow \lambda x.\lambda y.M$$

Ejemplos:

$$\lambda x.(\lambda y.x) \rightarrow \lambda x.\lambda y.x$$

$$\lambda a.(\lambda b.(\lambda c.a)) \rightarrow \lambda a.\lambda b.\lambda c.a$$

Convención 3: aplicación prioritaria sobre abstracción

$$(\lambda x.(MN)) \rightarrow \lambda x.MN$$

Ejemplos:

$$\lambda x.(x\ y) \rightarrow \lambda x.x\ y$$

$$\lambda x.((f\ x)\ z) \rightarrow \lambda x.f\ x\ z$$

Cuidado

$$\lambda x.M\ N \equiv \lambda x.(M\ N) \quad \text{y NO} \quad (\lambda x.M)\ N$$

Convención 4: suprimir λ en abstracciones consecutivas

$$\lambda x. \lambda y. \dots \lambda z. M \rightarrow \lambda xy \dots z. M$$

Ejemplos:

$$\lambda x. \lambda y. x \rightarrow \lambda xy. x$$

$$\lambda f. \lambda g. \lambda x. f (g x) \rightarrow \lambda f g x. f (g x)$$

Ejemplo integrador de lectura (sin reducir)

Expresión:

$$\lambda f g x. f (g x) y$$

Con paréntesis explícitos usando convenciones:

$$\lambda f. \lambda g. \lambda x. (((f (g x)) y))$$

- Aplicación a izquierda: $A B C \equiv ((A B) C)$.
- El cuerpo de $\lambda x.$ se extiende hasta el final (salvo paréntesis).

Variables libres y ligadas

- Una ocurrencia de x es **ligada** si está dentro del alcance de λx .
- Es **libre** si no está ligada por ningún λx .

Ejemplos:

$\lambda x. x y$ (x ligada, y libre)

$(\lambda x. x) y$ (y libre en el término completo)

Shadowing (mismo nombre, distinto alcance)

$$\lambda x. (\lambda x. x y) x$$

- El $\lambda x.$ interno **oculta** (shadowing) al x externo dentro de su cuerpo.
- El x final (argumento) pertenece al alcance del $\lambda x.$ externo.

Variables libres y ligadas

- En $\lambda x. E$, el símbolo x queda **ligado** (*bound*) dentro de E .
- Una ocurrencia de variable es **libre** si no está dentro del alcance de una λ que la ligue.
- Notación útil: $FV(E)$ (variables libres) y $BV(E)$ (variables ligadas).

Ejemplo 1:

$\lambda x. x y$

- x está ligada por λx
- y está libre

$FV(\lambda x. x y) = \{ y \}$

$BV(\lambda x. x y) = \{ x \}$

Variables libres y ligadas

Ejemplo 2:

$(\lambda x. \lambda y. x) z$

- x e y están ligadas en sus alcances
- z está libre en todo el término

$FV((\lambda x. \lambda y. x) z) = \{ z \}$

Reglas y Resolución

Las expresiones lambda no son muy interesantes por sí mismas. Lo que las hace interesantes son las muchas nociones de equivalencia y reducción que pueden definirse sobre ellas.

Semántica operacional

- La evaluación de una expresión se compone de pasos de reducción, donde cada uno de los pasos se obtiene por reescritura.
- Se parte de un estado inicial (expresión inicial) y, mediante un proceso de reescritura, se obtiene un estado final (expresión final).
- Cada reducción de e reemplaza cierta subexpresión de acuerdo con ciertas reglas; tales subexpresiones se llaman **redex** (*reducible expression*).

Forma normal

- Un término está en **forma normal** si **no contiene** redexes- β .
- Algunos términos **sí** tienen forma normal (y se puede llegar a ella).
- Otros términos **no** tienen forma normal (divergen), por ejemplo Ω .
- Teorema útil: si un término tiene forma normal, el **orden normal** la encuentra.

Ejemplo con forma normal:

$(\lambda x.x) y \rightarrow y$

Normal: y

Ejemplo sin forma normal:

$\Omega = (\lambda x.x x) (\lambda x.x x)$

No existe una forma normal (reduce infinitamente).

Regla de sustitución en cálculo lambda

- La **sustitución** se escribe $E[x := N]$ y significa: *reemplazar las ocurrencias libres* de x en E por el término N .
- La sustitución es el corazón de la **reducción- β** :

$$((\lambda x. E) N) \rightarrow_{\beta} E[x := N].$$

- Importante: la sustitución debe evitar **captura de variables** (cuando un nombre libre pasa a quedar ligado por accidente).

Beta-reducción:

$$((\lambda x. E) N) \rightarrow E[x := N]$$

Idea: sustituir **SOLO** las ocurrencias libres de x en E por N .

Reglas de sustitución (idea operativa)

- Queremos definir $E[x := N]$ respetando:
 - **Solo** se reemplazan ocurrencias **libres** de x .
 - Se debe evitar la **captura** de variables libres de N .
- Reglas típicas (informales):
 - 1 $x[x := N] = N$
 - 2 $y[x := N] = y$ si $y \neq x$
 - 3 $(E_1 E_2)[x := N] = (E_1[x := N] E_2[x := N])$
 - 4 $(\lambda y. E)[x := N] = \lambda y. E[x := N]$ si $y \neq x$ y $y \notin FV(N)$
- Si $y \in FV(N)$, antes se aplica **conversión- α** para renombrar y .

Captura de variables y conversión- α

- **Captura:** ocurre si, al sustituir, una variable libre del argumento queda accidentalmente ligada.
- **Solución: conversión- α** (renombrar variables ligadas) antes de sustituir.

Ejemplo de riesgo de captura:

$(\lambda x. \lambda y. x) y$

Si hacemos beta sin cuidado:

$((\lambda x. \lambda y. x) y) \rightarrow (\lambda y. y) \quad \leftarrow \text{¡capturó a } y!$

Pero la y del argumento era libre y terminó ligada por λy .

Captura de variables y conversión- α

Evitar captura con alfa:

$(\lambda x. \lambda y. x)$ y

 alfa: $(\lambda x. \lambda t. x)$ y (renombro la y ligada por t)

 beta: $(\lambda t. y)$

Resultado correcto: $\lambda t. y$

Reducción- β y ejemplos paso a paso

- La reducción- β aplica una función a un argumento:

$$((\lambda x. E) N) \rightarrow_{\beta} E[x := N].$$

- Repetimos el proceso hasta que ya no existan redexes- β (si es posible).

Ejemplo:

$(\lambda x. x) ((\lambda y. y) z)$

Paso 1 (reducir afuera):

$(\lambda x. x) A \rightarrow A$, con $A = ((\lambda y. y) z)$

$\Rightarrow ((\lambda y. y) z)$

Paso 2:

$(\lambda y. y) z \rightarrow z$

Forma normal: z

$$((\lambda x. \lambda y. x)y)z$$

$$\begin{aligned}
& (\lambda x y z. x y z)(\lambda x. x x)(\lambda x. x) x \\
\rightarrow_{\beta} & (\lambda y z. x y z)[x := \lambda x. x x](\lambda x. x) x \\
\equiv & (\lambda y z. (\lambda x. x x) y z)(\lambda x. x) x \\
\rightarrow_{\beta} & (\lambda y z. (x x)[x := y] z)(\lambda x. x) x \\
\equiv & (\lambda y z. y y z)(\lambda x. x) x \\
\rightarrow_{\beta} & (\lambda z. y y)[y := \lambda x. x] x \\
\equiv & (\lambda z. (\lambda x. x)(\lambda x. x) z) x \\
\rightarrow_{\beta} & (\lambda z. x[x := \lambda x. x] z) x \\
\equiv & (\lambda z. (\lambda x. x) z) x \\
\rightarrow_{\beta} & (\lambda z. x[x := z]) x \\
\equiv & (\lambda z. z) x \\
\rightarrow_{\beta} & z[z := x] \\
\equiv & x
\end{aligned}$$

Estrategias de reducción

- Un **redex**- β es una subexpresión de la forma $((\lambda x. E) N)$.
- La estrategia define **qué redex** reducimos primero.
- Las más usadas:
 - **Orden normal** (*leftmost-outermost*): reduce el redex más a la izquierda y más externo.
 - **Orden aplicativo** (*leftmost-innermost*): reduce primero argumentos (más interno).
 - **Call-by-name**: similar a orden normal, pero no reduce dentro de λ .
 - **Call-by-value**: similar a aplicativo: el argumento debe ser un valor.

Orden normal vs. orden aplicativo (ejemplo clásico)

- Definimos un término que **no termina**:

$$\Omega = (\lambda x. x x)(\lambda x. x x).$$

- Consideremos: $(\lambda x. z) \Omega$.
 - Con **orden normal** se llega a forma normal.
 - Con **orden aplicativo** se intenta reducir Ω primero y no termina.

Definición:

$\Omega = (\lambda x. x x) (\lambda x. x x)$

Término:

$(\lambda x. z) \Omega$

Orden normal vs. orden aplicativo (ejemplo clásico)

Orden normal (outermost):

$(\lambda x.z) \Omega \rightarrow z$

Forma normal: z

Orden aplicativo (innermost):

Primero intenta reducir Ω :

$(\lambda x.x x) (\lambda x.x x) \rightarrow (\lambda x.x x) (\lambda x.x x) \rightarrow \dots$ (no termina)

Nunca llega a z

Estrategias como reglas de elección

Orden normal (normal order)

Reduce **el redex más externo** y, entre ellos, **el más a la izquierda**.

Intuición: “*no evalúes argumentos si no hace falta*”.

Forma aplicativa (applicative order)

Reduce primero **dentro** (argumentos) antes de reducir el redex externo.

Intuición: “*evalúa argumentos antes de aplicar*”.

Ojo

Ambas usan β -reducción; difieren en **qué redex eligen** en cada paso.

Definiciones operativas (estilo algoritmo)

Normal order:

Repetir:

encontrar el redex outermost-leftmost
contraerlo

hasta que no haya redex (forma normal) o no termine

Applicative order:

Repetir:

encontrar el redex innermost-leftmost
contraerlo

hasta que no haya redex (forma normal) o no termine

- “Outermost” = no está contenido en otro redex.
- “Innermost” = no contiene ningún redex adentro (se reduce “lo más interno” primero).

Ejemplo 1: cuando ambas terminan (pero distinto camino)

Definimos:

$$I = \lambda x. x \quad K = \lambda x. \lambda y. x$$

Consideremos:

$$(K\ a)\ (I\ b)$$

Orden normal

$$(K\ a)\ (I\ b) \rightarrow_{\beta} (\lambda y. a)\ (I\ b) \rightarrow_{\beta} a$$

No necesita evaluar el argumento.

Forma aplicativa

$$(K\ a)\ (I\ b) \rightarrow_{\beta} (K\ a)\ b \rightarrow_{\beta} (\lambda y. a)\ b \rightarrow_{\beta} a$$

Evalúa primero $(I\ b)$.

Ejemplo 2: diferencia crítica en terminación

Definimos el término no terminante:

$$\Omega = (\lambda x. x x) (\lambda x. x x)$$

y el término:

$$(\lambda z. a) \Omega$$

Orden normal (termina)

$$(\lambda z. a) \Omega \rightarrow_{\beta} a$$

El argumento no se usa, entonces no se evalúa.

Forma aplicativa (diverge)

Primero intenta reducir Ω :

$$\Omega \rightarrow_{\beta} \Omega \rightarrow_{\beta} \Omega \rightarrow_{\beta} \dots$$

Nunca llega a aplicar $(\lambda z. a)$.

Teorema clave (idea)

Propiedad de orden normal

Si una λ -expresión **tiene** forma normal, entonces **el orden normal la encuentra**.

- Consecuencia: orden normal es **normalizante** (cuando existe normal form).
- Forma aplicativa **no** es normalizante en general (puede divergir aunque exista normal form).

Costo: duplicación de trabajo vs evaluación de argumentos

Ejemplo típico de duplicación:

$$(\lambda x. x x) E$$

- Orden normal: sustituye E en dos lugares $\Rightarrow$ puede **repetir** trabajo si E es caro.
- Forma aplicativa: suele evaluar E antes (una vez) $\Rightarrow$ evita duplicación, pero puede evaluar **de más** si x no se usa (como en el ejemplo con K).

Idea que conecta con lenguajes reales

Compartición (call-by-need, lazy con memoización) intenta combinar: *no evaluar si no hace falta + no repetir evaluaciones.*

Relación con estrategias en lenguajes

- **Call-by-name** $\approx$ orden normal (*sin compartir*).
- **Call-by-need** (lazy) $\approx$ orden normal *con compartir/memoización*.
- **Call-by-value** $\approx$ forma aplicativa restringida a *valores* (argumentos evaluados primero).

Advertencia

En λ puro hay varias variantes formales (weak head normal form, evaluación débil, etc.). La intuición general: **strict** $\leftrightarrow$ **aplicativa**, **lazy** $\leftrightarrow$ **normal**.

Cómo identificar “qué se reduce primero” (tips rápidos)

Dada una expresión, para **orden normal**:

- 1 Buscá la **aplicación más externa** cuyo operador sea una λ .
- 2 Si hay varias al mismo nivel, elegí la **más a la izquierda**.

Para **forma aplicativa**:

- 1 Metete en el árbol y buscá un redex **que no contenga** otro redex adentro.
- 2 Elegí el **más a la izquierda** entre esos.

Mini-ejercicio

Sea:

$$(\lambda x. \lambda y. x) ((\lambda u. u) v) \Omega$$

- Reducí 3 pasos con **orden normal**.
- Reducí 3 pasos con **forma aplicativa**.
- ¿En cuál estrategia aparece el problema de Ω primero?

Resumen

Orden normal

- Reduce **outermost-leftmost**.
- Si existe forma normal, **la encuentra**.
- Puede **duplicar trabajo** (sustituciones repetidas).

Forma aplicativa

- Reduce **innermost-leftmost**.
- Puede **divergir** aunque exista forma normal.
- Evita duplicación, pero puede evaluar **argumentos innecesarios**.

Uso

Cálculo lambda hoy: ¿por qué sigue vigente?

- El cálculo lambda es la base teórica de:
 - **Funciones anónimas** (lambdas) y **funciones de orden superior**.
 - **Clausuras** (closures) y captura de variables.
 - **Inmutabilidad** y composición funcional (*map*, *filter*, *fold*).
 - Modelos de evaluación (**call-by-value**, **call-by-name**, **lazy**).
- Aunque no se programa “en cálculo lambda puro”, sus ideas aparecen en casi todos los lenguajes modernos.

Idea central (lambda calculus):

- Abstracción: $\lambda x. E$
- Aplicación: $(E1\ E2)$

Versión moderna:

- "función anónima" + "aplicarla a un argumento"
-

Funciones lambda y funciones de orden superior (Java / JavaScript)

- **Funciones anónimas:** equivalentes prácticos de $\lambda x. E$.
- **Orden superior:** funciones que reciben o devuelven funciones (muy lambda-calculus).

```
import java.util.List;
```

```
List<Integer> xs = List.of(1,2,3,4);
```

```
// map: aplicar una lambda a cada elemento
```

```
var ys = xs.stream()  
    .map(x -> x * 2)  
    .toList();
```

```
// filter: seleccionar por predicado
```

```
var pares = xs.stream()  
    .filter(x -> x % 2 == 0)  
    .toList();
```

Composición funcional y pipelines de datos (Python)

- En ciencia de datos y software moderno es común:
 - transformar colecciones con **map/filter/reduce**
 - construir **pipelines** mediante composición
- Esto refleja el estilo “aplicar funciones a valores” (aplicación) y “crear funciones” (abstracción).

```
from functools import reduce
```

```
xs = [1,2,3,4,5,6]
```

```
pares = list(filter(lambda x: x % 2 == 0, xs))
```

```
dobles = list(map(lambda x: x * 2, pares))
```

```
suma = reduce(lambda acc, x: acc + x, dobles, 0)
```

```
print(pares)      # [2, 4, 6]
```

```
print(dobles)     # [4, 8, 12]
```

```
print(suma)       # 24
```

Patrones modernos: callbacks, eventos y asincronía (JavaScript)

- Interfaces gráficas, web y sistemas reactivos dependen de:
 - **callbacks** (funciones como valores)
 - **handlers** (funciones que se pasan a otras funciones)
- Es el “ADN” del cálculo lambda: funciones que viajan como datos.

```
// callback: una función pasada como argumento  
function fetchData(url, onSuccess, onError) {  
  // simula asincronía  
  setTimeout(() => onSuccess({ ok: true, url }), 100);  
}
```

```
fetchData(  
  "https://api.example.com/items",  
  (data) => console.log("OK:", data),  
  (err)  => console.error("ERR:", err)  
);
```

Un puente explícito: del cálculo lambda al código moderno

- Ejemplo conceptual: aplicar una función a un argumento.

Cálculo lambda:

$((\lambda x. x + 1) 41) \rightarrow 42$

Lectura:

- $\lambda x. (x + 1)$ es una función
- aplicada a 41

Python (equivalente práctico)

```
f = lambda x: x + 1  
print(f(41)) # 42
```

Un puente explícito: del calculo lambda al codigo moderno

- Ejemplo conceptual: aplicar una funcion a un argumento.

```
import java.util.function.IntUnaryOperator;  
  
IntUnaryOperator f = x -> x + 1;  
System.out.println(f.applyAsInt(41)); // 42
```
